Data Analysis Textbook

SQL app　Excel Microsoft 365　Python pandas

データ分析の教科書

第 1 部〜第 5 部

Part 1–5 · Unit A1–A14

数字の裏にある事実を、3 つの道具で確かめる。

Find the facts behind the numbers, and check them with three tools.

kumo_mart.sql

```sql
-- How much does a typical order cost?
SELECT ROUND(AVG(amount), 1) AS mean,
       (SELECT amount FROM kumo_orders
        ORDER BY amount LIMIT 1 OFFSET 200) AS median
FROM kumo_orders;
```

結果 1 row

mean	median
5351.4	4390

No.	名前 Name

この本の使い方 How to use this book

1 つの Unit は見開き 2 ページ。アプリで SQL を書き、同じ問いを Excel と Python でも解く。

Each unit is a two-page spread. Write the SQL in the app, then solve the same question in Excel and Python.

左のページ left page

① 依頼とゴール

Request and goal

クモマートの誰かからの依頼と、

この Unit でできるようになること。

A request from someone at Kumo Mart, and what you will be able to do.

② 統計の基礎

Statistics basics

式の意味を、本当のデータで描いた図で見る。

The meaning of the formula, with a figure drawn from the real data.

③ 見てみる

See it

クモマートのデータで SQL を実行した結果。

The SQL run on the Kumo Mart data, and its result.

右のページ right page

④ ポイント・よくある間違い

Points, common mistakes

× の SQL を実行するとどうなるか。

What happens when the × SQL runs.

⑤ Excel・Python

Excel, Python

同じ問いを Excel と Python でも解く。

同じ数字が出れば正解。

Solve the same question in Excel and Python. The same numbers mean you are right.

⑥ やってみよう

Try it

別の業種のデータで練習。答えは巻末。

Practise on data from other businesses. Answers are at the back.

分析の流れ The analysis process

問う Ask → 集める Collect → 整える Clean → 分析する Analyse → 伝える Share

どの Unit も、この流れのどこかを練習している。用語は英語も一緒に覚えると、英語の資格試験もそのまま読める。

Every unit practises part of this process. Learn the English terms too, so you can read English certification exams.

CSV と道具 CSV and tools

SQL

アプリの「データ分析」の章。

問題の下のボタンで Excel・Python のガイドが開く。

The “Data analysis” chapter of the app. The buttons under each problem open the Excel and Python guides.

Py

Python は VS Code か Jupyter。

pandas・matplotlib を使う。

ガイドからノートブックも保存できる。

Python runs in VS Code or Jupyter with pandas and matplotlib. The guide can also save a notebook.

X

Excel は Microsoft 365。

表の「CSV」ボタンで保存したファイルを開く。

Excel is Microsoft 365. Open the file saved with the CSV button of the table.

各 Unit のページの QR コードで、アプリのその Unit が開く。

The QR code on each unit opens that unit in the app.

講座の地図 Course map

この本は 14 Unit・5 部。記述統計から、比較・推移・相関、そして結果を読んで決めるところまで。
灰色は次の巻で学ぶ。

14 units in 5 parts: from descriptive statistics to comparing, trends, correlation, and reading results to decide. Grey topics come in the next volume.

第 1 部
データと向き合う準備
Part 1 · Getting ready

- **A1** データの全体を見る — Get to know the data
- **A2** 欠損値と重複 — Missing values, duplicates

第 2 部
1 つの数値を要約する
Part 2 · Summarise one number

- **A3** 平均と中央値 — Mean, median, mode
- **A4** 散らばり — Spread
- **A5** 度数分布 — Histograms
- **A6** 標準化と外れ値 — z-scores, outliers

第 3 部
比べる・組み合わせる
Part 3 · Compare and combine

- **A7** クロス集計と構成比 — Cross tables, shares
- **A10** グループの比較 — Compare groups
- **A11** 表を合わせる — Joining tables
- **A12** データの下ごしらえ — Getting data ready

第 4 部
時間と関係
Part 4 · Time and relationships

- **A8** 月ごとの推移 — Monthly trends
- **A9** 相関 — Correlation

第 5 部
読む・決める
Part 5 · Read and decide

- **A13** 結果を正しく読む — Reading results
- **A14** 総合演習 — Case study

第 6 部
この先（次の巻）
Part 6 · Next volume

- – 回帰と予測 — Regression
- – 推定 — Estimation
- – A/B テスト — A/B tests
- – カイ二乗検定 — Chi-square test

この本でできるようになること What you will be able to do

できるようになったら、□ にチェックを入れよう。

Tick each box when you can do it.

- ☐ 列の種類（量的・質的）を言える — name the kind of each column — A1
- ☐ 欠損率と重複を数えて報告できる — report missing values and duplicates — A2
- ☐ 平均・中央値・最頻値を使い分けられる — choose between mean, median and mode — A3
- ☐ 標準偏差で散らばりを比べられる — compare spread with the standard deviation — A4
- ☐ 度数分布表を作り、形を読める — make a frequency table and read its shape — A5
- ☐ z 得点で外れ値の候補を見つけられる — find outlier candidates with z-scores — A6
- ☐ クロス集計を作り、% で比べられる — make a cross table and compare in % — A7
- ☐ 月ごとの推移を前月差・移動平均で読める — read monthly trends with changes and moving averages — A8
- ☐ 相関係数を出し、因果と区別できる — find r and tell it from causation — A9
- ☐ グループを比べ、グループごとの順位を付けられる — compare groups and rank inside each group — A10
- ☐ 2 つの表を JOIN して集計できる — JOIN two tables and aggregate — A11
- ☐ 書き方のゆれ・空欄・区分を整えられる — tidy spellings, blanks and bands — A12
- ☐ 結果を正しく読み取れる — read results correctly — A13
- ☐ 分析して、数字を理由に決められる — analyse and decide with the numbers as reasons — A14

目次 Contents

データと向き合う準備 Part 1 · Getting ready

Unit A1 データの全体を見る Get to know the data p.6

Unit A2 欠損値と重複を数える Count missing values and duplicates p.8

1 つの数値を要約する（記述統計） Part 2 · Summarise one number (descriptive statistics)

Unit A3 平均と中央値 Mean, median and mode p.10

Unit A4 散らばりを測る Measure the spread p.12

Unit A5 度数分布とヒストグラム Frequency tables and histograms p.14

Unit A6 標準化と外れ値 Standard scores and outliers p.16

比べる・時間・関係 Parts 3–4 · Compare, over time, relationships

Unit A7 クロス集計と構成比 Cross tables and shares p.18

Unit A8 月ごとの推移 Month by month p.20

Unit A9 2 つの数の関係（相関） Relationship between two numbers p.22

Unit A10 グループを比べる Comparing groups p.24

Unit A11 2 つの表を合わせて分析する Analysing two tables together p.26

Unit A12 データの下ごしらえ Getting data ready p.28

読む・決める Part 5 · Read and decide

Unit A13 結果を正しく読む Reading results correctly p.30

Unit A14 次のキャンペーンはどの店で？ Where should the next campaign be? p.32

答え Answers

やってみようの答え Answers to “Try it” p.34

クモマートのデータ The Kumo Mart data

クモマート（Kumo Mart）は、3 つの店とネット通販を持つ架空の会社。例題はすべてこのデータを使う。
アプリと同じデータなので、結果を確かめられる。

Kumo Mart is a made-up company with three stores and an online shop. All examples use its data, which is the same as in the app.

`kumo_orders` 注文（全部で 401 行、最初の 8 行） orders (401 rows, the first 8)

order_id	order_date	store	customer_id	amount	items	coupon	stay_min
1	2025-01-01	Sakae	C102	2090	4	SPRING5	26
2	2025-01-02	Online	C042	7110	2	*NULL*	30
3	2025-01-03	Online	C081	2880	5	WELCOME10	7
4	2025-01-03	Sakae	C034	4660	1	APP300	21
5	2025-01-03	Umeda	C031	2090	6	*NULL*	9
6	2025-01-05	Online	C104	7690	1	*NULL*	22
7	2025-01-07	Sakae	C088	5390	1	*NULL*	56
8	2025-01-08	Shibuya	C089	7560	2	*NULL*	42

order_id	注文番号 order number
order_date	注文日（2025 年） order date (2025)
store	店（Shibuya・Umeda・Sakae・Online） store
customer_id	客の番号（同じ客が何度も買う） customer (repeats)
amount	金額（円） amount (yen)
items	買った点数 number of items
coupon	使ったクーポン（空欄あり） coupon used (may be blank)
stay_min	店・サイトにいた時間（分） minutes in the store / site

顧客名簿 kumo_customers は A2、会員の表 kumo_accounts（customer_id・joined_date・plan・age・pref）は A11・A12 で使う。
pref は手入力で、書き方がばらばら。

The customer list kumo_customers is used in A2; the member table kumo_accounts (customer_id, joined_date, plan, age, pref) in A11 and A12. pref is hand-typed and messy.

練習で使う表 Tables for practice

「やってみよう」とアプリの演習では、別の業種のデータで同じ方法を使う。

In “Try it” and in the app, the same methods are used on data from other businesses.

各業種には人の表もある（cafe_members・hotel_guests・courier_clients・helpdesk_users・gym_members・factory_staff）。
Each business also has a people table (cafe_members, hotel_guests, courier_clients, helpdesk_users, gym_members, factory_staff).

表 table	内容 about	行数 rows	列 columns
cafe_sales	カフェの売上 cafe sales	151	`sale_id, sale_date, shop, member_id, amount, cups, coupon, stay_min`
hotel_stays	ホテルの宿泊 hotel stays	131	`stay_id, checkin_date, hotel, guest_id, charge, nights, request, room_m2`
courier_jobs	宅配の料金 courier jobs	141	`delivery_id, ship_date, region, client_id, fee, days, note, weight_kg`
helpdesk_tickets	IT ヘルプデスクの対応 IT help desk tickets	121	`ticket_id, opened_date, team, user_id, minutes, replies, rating, score`
gym_visits	ジムの利用 gym visits	161	`visit_id, visit_date, branch, member_id, minutes, machines, trainer, kcal`
factory_lots	工場の製品検査 factory quality checks	111	`lot_id, made_date, line, operator_id, weight_g, defects, inspector, temp_c`

分析 Unit A1

データの全体を見る COUNT・MIN・MAX

Get to know the data

アプリで練習
Practice in the app

SQL Excel Python

店長 Store manager

「最近、売上が落ちた気がする。」答える前に、どんなデータがあるかを確かめよう。

"I feel sales have dropped lately." Before answering, check what data we have.

行数・値の範囲・期間を調べ、列の種類を言える。

You can find the number of rows, the range of values and the period, and name the kind of each column.

統計の基礎 Statistics basics

データの種類 Kinds of data

- 量的データ：計算できる数（金額、個数）
  Quantitative: numbers you can calculate with (amount, items)
- 質的データ：ラベル（店、クーポン）。名義尺度は順番なし、順序尺度は順番あり（S・M・L）
  Qualitative: labels (store, coupon). Nominal has no order, ordinal has an order (S, M, L)
- 日付：順番があり、日数を数えられる
  Dates: they have an order, and you can count days between them

母集団と標本 Population and sample

母集団は知りたいもの全部（すべての客）、
標本は手元にある一部（今年の注文）。統計は、
標本から母集団を知るための道具。

The population is everything you want to know about (all customers); a sample is the part you have (this year's orders). Statistics learns about the population from a sample.

order_id	ID id
order_date	日付 date
store	質的 qualitative
customer_id	質的 qualitative
amount	量的 quantitative
items	量的 quantitative
coupon	質的 qualitative
stay_min	量的 quantitative

kumo_orders の列と、その種類。

The columns of kumo_orders and their kinds.

見てみる See it

kumo_orders の行数、amount の最小と最大、最初と最後の日付を求める。

Find the number of rows, the smallest and largest amount, and the first and last date of kumo_orders.

```sql
SELECT COUNT(*) AS n_rows,
       MIN(amount) AS min_amount, MAX(amount) AS max_amount,
       MIN(order_date) AS first_date, MAX(order_date) AS last_date
FROM kumo_orders;
```

結果 1行 1 row

n_rows	min_amount	max_amount	first_date	last_date
401	820	68400	2025-01-01	2025-12-31

2025 年の 1 年分、401 件の注文。金額の幅はとても広い。

One year (2025), 401 orders. The amounts vary a lot.

ポイント Points

1. 分析の流れ：問う → 集める → 整える → 分析する → 伝える。この Unit は「集める」の最初の 1 歩。
   The analysis process: ask → collect → clean → analyse → share. This unit is the first step of "collect".
2. いきなり細かく見ない。まず行数・範囲・期間で全体をつかむ。
   Do not start with the details. First get the whole picture: rows, range, period.
3. 日付は '2025-01-31' の形の文字なので、MIN と MAX で最初と最後が分かる。
   Dates are text like '2025-01-31', so MIN and MAX give the first and last.

よくある間違い Common mistakes

✕
```
SELECT COUNT(coupon)
FROM kumo_orders;
```

実行結果 1行 1 row

COUNT(coupon)
~~401~~
143

→ 結果の値がちがう
→ different values in the result

COUNT(列) は NULL を数えない。行数は COUNT(*) で数える。
COUNT(column) does not count NULL. Count rows with COUNT(*).

◯
```
SELECT COUNT(*)
FROM kumo_orders;
```

X Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 行数、amount の最小と最大：
   Rows, smallest and largest amount:
```
=ROWS(kumo_orders)
=MIN(kumo_orders[amount])
=MAX(kumo_orders[amount])
```
3. 最初と最後の日付。45658 のような数が出たら、表示形式を「日付」にする。
   First and last date. If you see a number like 45658, set the number format to Date.
```
=MIN(kumo_orders[order_date])
=MAX(kumo_orders[order_date])
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Py Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.
```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```
2. 行数・範囲・期間。df.info() で列ごとの型も分かる。
   Rows, range, period. df.info() also shows the type of each column.
```
print(len(df))
print(df["amount"].min(), df["amount"].max())
print(df["order_date"].min(),
df["order_date"].max())
df.info()
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. cafe_sales の行数と、amount の最小・最大を求めよう。
   Find the number of rows and the smallest and largest amount of cafe_sales.
   結果：1行 1 row

2. hotel_stays の checkin_date の最初と最後を求めよう。
   Find the first and last checkin_date of hotel_stays.
   結果：1行 1 row

3. gym_visits の列を、量的・質的・日付・ID に分けよう。
   Sort the columns of gym_visits into quantitative, qualitative, date and ID.
   列は 8 つ 8 columns

分析
Unit
A2

欠損値と重複を数える

COUNT(列)・DISTINCT

Count missing values and duplicates

アプリで練習
Practice in the app

SQL Excel Python

マーケティング部
Marketing team

「会員にメールを送りたい。何人に送れる？」名簿には空欄と二重登録がある。

“We want to e-mail our members. How many can we reach?” The list has gaps and double sign-ups.

空欄（NULL）と重複の数を数え、割合で報告できる。

You can count missing values (NULL) and duplicates and report them as a percentage.

統計の基礎 Statistics basics

実際のデータには必ず空欄がある。欠損率は、
空欄の数を全体の行数で割る。

Real data always has gaps. The missing rate is the number of blanks divided by all rows.

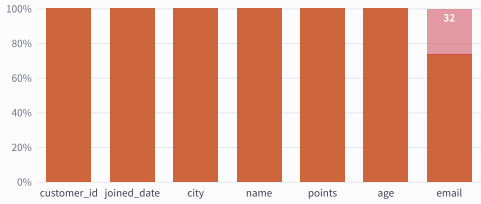
欠損率 (%) = (COUNT(*) − COUNT(列)) ÷ COUNT(*) × 100

どうするかは、なぜ空欄かで決める。
行を除く・値を埋める・「不明」として残す。除いた数は必ず報告する。

What to do depends on why it is missing: drop the row, fill it in, or keep “unknown”. Always report how many you dropped.

同じ人が 2 回登録されることもある。
COUNT(DISTINCT 列) は同じ値を 1 回だけ数える。

The same person may sign up twice. COUNT(DISTINCT column) counts each value once.



kumo_customers の列ごとの、
値が入っている割合（青）と空欄（赤、数字は件数）。

Per column of kumo_customers: filled (blue) and missing (red, with the count).

見てみる See it

kumo_customers の email について、入力あり・空欄・欠損率を求める。

For email in kumo_customers, find filled, missing and the missing rate.

```sql
SELECT COUNT(*) AS n_rows,
       COUNT(email) AS filled,
       COUNT(*) - COUNT(email) AS missing,
       ROUND(100.0 * (COUNT(*) - COUNT(email)) / COUNT(*), 1) AS missing_pct
FROM kumo_customers;
```

結果 1行 1 row

n_rows	filled	missing	missing_pct
121	89	32	26.4

メールを送れるのは 89 人…ではない。二重登録があるので、次に重複を数える。

We cannot simply e-mail 89 people: there are double sign-ups, so count duplicates next.

重複を数える Count duplicates

```sql
SELECT COUNT(name) AS n_rows,
       COUNT(DISTINCT name) AS unique_name
FROM kumo_customers;
```

n_rows	unique_name
121	98

121 行あるのに、名前は 98 種類。差の 23 行は、同じ人かどうか確かめる。

121 rows but only 98 different names. Check whether the other 23 rows are the same people.

ポイント Points

1. COUNT(*) は全部の行、COUNT(列) は NULL でない行を数える。差が空欄の数。
   COUNT(*) counts all rows, COUNT(column) the non-NULL ones. The difference is the number of blanks.
2. 割合は 100.0 を掛けてから割る。小数の計算になる。
   For a percentage, multiply by 100.0 first, so the division keeps decimals.

よくある間違い Common mistakes

✕
```
SELECT ROUND(100 * (COUNT(*) - COUNT(email)) / COUNT(*), 1)
FROM kumo_customers;
```

実行結果 1行 1 row

ROUND(100 * (COUNT(*) - COUNT(email)) / COUNT(*), 1)
~~26.4~~
26

→ 結果の値がちがう
→ different values in the result

整数どうしの割り算は、小数が切り捨てられる。
Dividing an integer by an integer drops the decimals.

○
```
SELECT ROUND(100.0 * (COUNT(*) - COUNT(email)) / COUNT(*), 1)
FROM kumo_customers;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、
   名前を kumo_customers にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_customers.
2. 行数・入力あり・空欄・欠損率：
   Rows, filled, blank, missing rate:
```
=ROWS(kumo_customers)
=COUNTA(kumo_customers[email])
=COUNTBLANK(kumo_customers[email])
=ROUND(100*COUNTBLANK(kumo_customers[email])/ROWS(
kumo_customers),1)
```
3. 重複を除いた名前の数（UNIQUE は Microsoft 365）：
   Different names (UNIQUE needs Microsoft 365):
```
=COUNTA(UNIQUE(kumo_customers[name]))
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.
```
import pandas as pd
df = pd.read_csv("kumo_customers.csv")
```
2. 空欄の数（全部の列）と、名前の種類：
   Blanks in every column, and the number of different names:
```
print(df.isna().sum())
missing = df["email"].isna().sum()
print(round(100 * missing / len(df), 1))
print(df["name"].nunique())
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. helpdesk_tickets の rating の空欄の数と欠損率（%、小数第 1 位）を求めよう。
   Find the number of blanks and the missing rate (%, 1 decimal place) of rating in helpdesk_tickets.
   結果：1 行 / 1 row

2. courier_jobs の client_id は何種類ある？
   How many different client_id are there in courier_jobs?
   結果：1 行 / 1 row

3. cafe_sales で、一度でも来た member_id の数と、全部の行数を並べて出そう。
   Show the number of different member_id next to the number of rows in cafe_sales.
   結果：1 行 / 1 row

分析 Unit A3

平均と中央値 AVG・中央値

Mean, median and mode

アプリで練習
Practice in the app

SQL Excel Python

店長 Store manager

「客単価はいくら？高額の注文に引っ張られていない？」

“How much does a customer spend per order? Are a few big orders pulling the number up?”

平均・中央値・最頻値を求め、どれが「ふつうの値」に近いかを説明できる。

You can find the mean, median and mode and explain which is closer to a typical value.

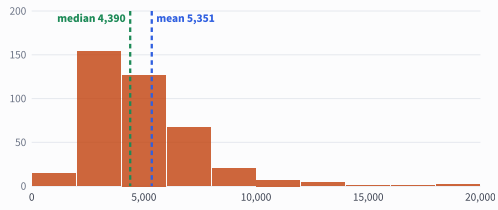
統計の基礎 Statistics basics

- 平均（mean）：合計 ÷ 個数
  Mean: the sum ÷ the number of values
- 中央値（median）：小さい順に並べた真ん中。
  n 個なら (n + 1) ÷ 2 番目
  Median: the middle value when sorted; with n values, the (n + 1) ÷ 2-th
- 最頻値（mode）：いちばん多く出てくる値
  Mode: the value that appears most often

平均 $\bar{x} = (x_1 + x_2 + \dots + x_n) \div n$

とても大きな値が少しあると、平均は引っ張られるが、
中央値は動かない。お金のデータは右に長いすそがあり、
平均が中央値より大きくなりやすい。

A few very large values pull the mean up, but not the median. Money data has a long right tail, so the mean is usually larger than the median.



amount のヒストグラム。青が平均、緑が中央値。
20,000 円以上の 4 件（最大 68,400 円）は図の外。

Histogram of amount: mean (blue), median (green). 4 orders of 20,000 yen or more (up to 68,400) are beyond the right edge.

見てみる See it

amount の平均と中央値を求める。401 行なので、中央値は小さい順で 201 番目。

Find the mean and median of amount. With 401 rows, the median is the 201th value.

```sql
SELECT ROUND(AVG(amount), 1) AS mean_amount,
       (SELECT amount FROM kumo_orders
        ORDER BY amount LIMIT 1 OFFSET 200) AS median_amount
FROM kumo_orders;
```

結果 1 行 1 row

mean_amount	median_amount
5351.4	4390

平均 5,351.4 円は、中央値 4,390 円よりかなり大きい。客単価の「ふつう」は中央値で伝える。

The mean (5,351.4 yen) is much larger than the median (4,390 yen). Report the typical order with the median.

ポイント Points

1. 括弧の中の SELECT（サブクエリ）が、中央値を 1 つの値として返す。
   The SELECT inside the brackets (a subquery) returns the median as one value.
2. OFFSET k は「k 行とばす」。真ん中の前にある (n − 1) ÷ 2 行をとばす。
   OFFSET k skips k rows: skip the (n − 1) ÷ 2 rows before the middle one.
3. 最頻値は GROUP BY で数えて、多い順の 1 行目。items なら 3（115 件）。
   For the mode, count with GROUP BY and take the first row. For items: 3 (115 orders).

```sql
SELECT items, COUNT(*) AS n FROM kumo_orders GROUP BY items ORDER BY n DESC LIMIT 1;
```

よくある間違い Common mistakes

✕
```
SELECT amount FROM kumo_orders
LIMIT 1 OFFSET 200;
```

実行結果 1行 1 row

amount
~~4390~~
6820

→ 結果の値がちがう
→ different values in the result

ORDER BY がないと、行は金額の順に並ばない（ここでは日付順）。真ん中の行は、並べてから取る。
Without ORDER BY the rows are not sorted by amount (here they are in date order). Sort first, then take the middle row.

○
```
SELECT amount FROM kumo_orders
ORDER BY amount LIMIT 1 OFFSET 200;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、
   Excel で開く。Ctrl + T でテーブルにして、
   名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 平均（小数第 1 位）・中央値・最頻値：
   Mean (1 decimal place), median and mode:

```
=ROUND(AVERAGE(kumo_orders[amount]),1)
=MEDIAN(kumo_orders[amount])
=MODE.SNGL(kumo_orders[items])
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas・VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 平均・中央値・最頻値と、ヒストグラム：
   Mean, median, mode and a histogram:

```
print(round(df["amount"].mean(), 1),
df["amount"].median())
print(df["items"].value_counts().head(1))
df["amount"].plot.hist(bins=30)
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. hotel_stays（131 行）の charge の平均と中央値を求めよう。
   Find the mean and median of charge in hotel_stays (131 rows).
   結果：1 行 1 row

2. gym_visits（161 行）の minutes の中央値を求めよう。
   Find the median of minutes in gym_visits (161 rows).
   結果：1 行 1 row

3. factory_lots の defects の最頻値と、その件数を求めよう。
   Find the mode of defects in factory_lots and how many rows have it.
   結果：1 行 1 row

分析 Unit A4

散らばりを測る 標準偏差

Measure the spread

アプリで練習
Practice in the app

SQL Excel Python

エリアマネージャー Area manager

「売上が安定している店はどこ？」平均だけでは分からない。

“Which store has steady sales?” The mean alone cannot tell you.

範囲・四分位数・標準偏差で散らばりを比べられる。

You can compare spread with the range, quartiles and the standard deviation.

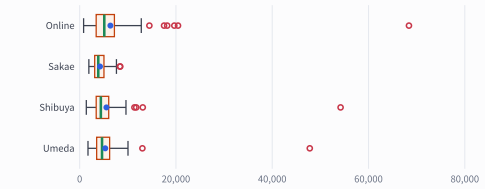
統計の基礎 Statistics basics

- 範囲（range）＝最大値 − 最小値。極端な値 1 つで大きく変わる
  Range = max − min. One extreme value changes it a lot
- 四分位数：Q1（25%）・中央値・Q3（75%）。IQR = Q3 − Q1 は真ん中半分の幅
  Quartiles: Q1 (25%), median, Q3 (75%). IQR = Q3 − Q1 is the width of the middle half
- 標準偏差（standard deviation）：平均から、だいたいどのくらい離れているか
  Standard deviation: roughly how far values are from the mean

標本分散 $s^2 = \Sigma(x - \bar{x})^2 \div (n - 1)$　　**標準偏差** $s = \sqrt{s^2}$

SQL では同じ値を $\Sigma(x-\bar{x})^2 = \Sigma x^2 - (\Sigma x)^2 \div n$ で計算する。n − 1 で割るのは、Excel の STDEV.S、pandas の .std() と同じ。

In SQL we use $\Sigma(x-\bar{x})^2 = \Sigma x^2 - (\Sigma x)^2 \div n$. Dividing by n − 1 is the same as Excel STDEV.S and pandas .std().



店ごとの箱ひげ図。箱が Q1〜Q3、緑の線が中央値、点が平均、丸は外れ値。

Box plot per store: box = Q1–Q3, green line = median, dot = mean, circles = outliers.

見てみる See it

店ごとに、件数・平均・標準偏差を求め、標準偏差の小さい順に並べる。

For each store, find the count, mean and standard deviation, smallest standard deviation first.

```sql
SELECT store, COUNT(*) AS n,
       ROUND(AVG(amount), 1) AS mean_amount,
       ROUND(SQRT((SUM(amount * amount) - 1.0 * SUM(amount) * SUM(amount) / COUNT(*))
                  / (COUNT(*) - 1)), 1) AS sd_amount
FROM kumo_orders
GROUP BY store
ORDER BY sd_amount;
```

結果 4 行 4 rows

store	n	mean_amount	sd_amount
Sakae	112	4221.4	1508.8
Umeda	94	5315.6	4829.2
Shibuya	87	5560.6	5752
Online	108	6385.8	7067.8

Sakae は標準偏差がいちばん小さく（1,508.8）、売上が安定している。Online はいちばん散らばっている。

Sakae has the smallest standard deviation (1,508.8): the steadiest. Online varies the most.

ポイント Points

1. 平均が違うグループは、変動係数 = s ÷ $\bar{x}$ で比べるとよい。
   For groups with different means, compare the coefficient of variation = s ÷ $\bar{x}$.
2. 範囲は MAX − MIN で求められる。
   The range is MAX − MIN.
   `SELECT store, MAX(amount) - MIN(amount) AS range_amount FROM kumo_orders GROUP BY store;`

よくある間違い Common mistakes

```sql
SELECT ROUND(SQRT((SUM(amount * amount) - 1.0 * SUM(amount) * SUM(amount) / COUNT(*))
                  / COUNT(*)), 1) AS sd
FROM kumo_orders;
```

実行結果 1 行 1 row

sd
5214
5207.5

→ **結果の値がちがう**
→ different values in the result

n で割ると母集団の式（STDEV.P）になる。標本は n − 1 で割る。
Dividing by n is the population formula (STDEV.P). For a sample, divide by n − 1.

```sql
SELECT ROUND(SQRT((SUM(amount * amount) - 1.0 * SUM(amount) * SUM(amount) / COUNT(*))
                  / (COUNT(*) - 1)), 1) AS sd
FROM kumo_orders;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 挿入 → ピボットテーブル。store を「行」、amount を 3 回「値」へ。
   値フィールドの設定で個数・平均・標本標準偏差にする。
   Insert → PivotTable. store to Rows, amount three times to Values; set them to Count, Average and StdDev.
3. 1つの店だけなら式でもよい：
   For one store you can also use a formula:

```
=ROUND(STDEV.S(FILTER(kumo_orders[amount],kumo_orders[store]="Sakae")),1)
```

4. 箱ひげ図：挿入 → 統計グラフ → 箱ひげ図。
   Box plot: Insert → Statistic Chart → Box and Whisker.

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 店ごとの件数・平均・標準偏差と、箱ひげ図：
   Count, mean and standard deviation per store, and a box plot:

```
df.groupby("store")["amount"].agg(["count", "mean", "std"]).round(1).sort_values("std")
df.boxplot(column="amount", by="store")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 helpdesk_tickets の team ごとに、minutes の標準偏差を求めよう。
Find the standard deviation of minutes for each team in helpdesk_tickets.
結果：3 行
3 rows

2 cafe_sales の shop ごとに、amount の範囲（最大 − 最小）を求めよう。
Find the range (max − min) of amount for each shop in cafe_sales.
結果：3 行
3 rows

3 factory_lots の line ごとに weight_g の標準偏差を求め、いちばん安定したラインを答えよう。
Find the standard deviation of weight_g per line in factory_lots. Which line is the most stable?
結果：3 行
3 rows

分析 Unit A5

度数分布とヒストグラム FLOOR・GROUP BY

Frequency tables and histograms

アプリで練習
Practice in the app

SQL Excel Python

店長 Store manager

「どのくらいの金額の注文がいちばん多い？」値を区切って数えよう。

“What size of order is the most common?” Split the values into ranges and count.

階級を決めて度数分布表を作り、ヒストグラムの形を読める。

You can make a frequency table with bins and read the shape of a histogram.

統計の基礎 Statistics basics

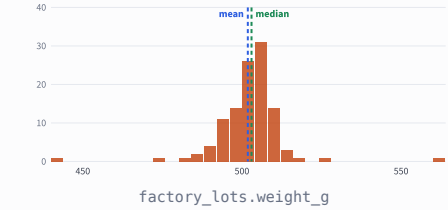
階級（bin）の幅を決め、各階級の行数（度数）を数える。
全体に対する割合が相対度数。

Choose a bin width and count the rows in each bin (frequency). The share of all rows is the relative frequency.

```
階級の始まり = FLOOR(x ÷ 幅) × 幅
```

形の読み方 Reading the shape

- 左右対称で山が１つ：平均 ≈ 中央値（製品の重さ）
  Symmetric, one peak: mean ≈ median (weight of products)
- 右に長いすそ：平均 > 中央値（お金、待ち時間）
  Long right tail: mean > median (money, waiting time)
- 山が２つ：違うグループが混ざっている。分けてから見る
  Two peaks: two groups are mixed — split them and look again





左は左右対称、右は右に長いすそ。
右では平均（青）が中央値（緑）より右にある。

Left: symmetric. Right: long right tail, where the mean (blue) is to the right of the median (green).

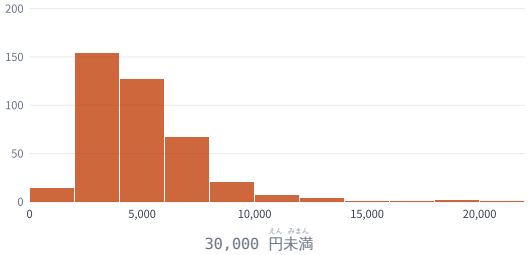
見てみる See it

amount を 2,000 円ずつの階級に分け、階級ごとの件数を数える。

Split amount into bins of 2,000 yen and count the orders in each bin.

```sql
SELECT FLOOR(1.0 * amount / 2000) * 2000 AS bin,
       COUNT(*) AS n
FROM kumo_orders
GROUP BY bin
ORDER BY bin;
```

bin	n	bin	n
0	14	14000	1
2000	154	16000	1
4000	127	18000	2
6000	67	20000	1
8000	20	46000	1
10000	7	54000	1
12000	4	68000	1



bin は階級の始まり。2000 は「2,000 円以上 4,000 円未満」。

bin is the start of the bin: 2000 means “2,000 to under 4,000 yen”.

ポイント Points

1. 相対度数（%）は、全体の件数をサブクエリで求めて割る。
   For the relative frequency (%), divide by the total count from a subquery.
   ```sql
   SELECT items, COUNT(*) AS n, ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM kumo_orders), 1) AS pct FROM kumo_orders GROUP BY items;
   ```
2. 幅が広すぎると形が消え、狭すぎるとギザギザになる。√n 個くらいの階級が目安。
   Too wide hides the shape, too narrow looks noisy. About √n bins is a good start.

よくある間違い Common mistakes

```sql
SELECT amount, COUNT(*) AS n
FROM kumo_orders
GROUP BY amount;
```

実行結果 313 行 313 rows

amount	n
0	14
2000	154
4000	127
6000	67
8000	20
10000	7

… 全部で 313 行

→ 313 行になる（正しくは 14 行）

→ 313 rows (correct: 14 rows)

金額そのものでまとめると、ほとんどの行が 1 件ずつになり、形が見えない。

Grouping by the amount itself gives almost one row per order, and you cannot see the shape.

```sql
SELECT FLOOR(1.0 * amount / 2000) * 2000 AS bin, COUNT(*) AS n
FROM kumo_orders
GROUP BY bin;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. テーブルの右に bin 列を足し、
   ピボットテーブルで bin を「行」、amount の個数を「値」へ。
   Add a bin column, then a PivotTable: bin to Rows, Count of amount to Values.

```
=FLOOR.MATH([@amount]/2000)*2000
```

3. ヒストグラム：amount の列を選び、
   挿入 → 統計グラフ → ヒストグラム。
   ビンの幅を 2000 にする。
   Histogram: select amount → Insert → Statistic Chart → Histogram; set the bin width to 2000.

SQL と同じ数字になれば正解。

If you get the same numbers as the SQL, you are right.

Python でやってみる Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 度数分布表とヒストグラム：
   Frequency table and histogram:

```
df["bin"] = (df["amount"] // 2000) * 2000
print(df.groupby("bin").size())
import numpy as np
df["amount"].plot.hist(bins=np.arange(0, 70001, 2000))
```

SQL と同じ数字になれば正解。

If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 courier_jobs の fee を 500 円ずつの階級に分けて数えよう。
Count the fee of courier_jobs in bins of 500 yen.

結果：8 行
8 rows

2 hotel_stays の nights ごとの件数と相対度数（%）を求めよう。
Find the count and relative frequency (%) of each nights value in hotel_stays.

結果：6 行
6 rows

3 factory_lots の weight_g を 5 g ずつの階級に分けて数え、形を言葉で説明しよう。
Count weight_g of factory_lots in bins of 5 g and describe the shape in words.

結果：12 行
12 rows

分析 Unit A6

標準化と外れ値 z 得点

Standard scores and outliers

アプリで練習
Practice in the app

SQL Excel Python

経理部 Accounting

「おかしな注文がないか確かめて。」平均からどれだけ離れているかで探そう。

"Check for strange orders." Look for values far from the mean.

z 得点を計算し、外れ値の候補を見つけて確かめられる。

You can calculate z-scores and find outlier candidates to check.

統計の基礎 Statistics basics

```
z = (x − x̄) ÷ s      偏差値 (T-score) = 50 + 10z
```

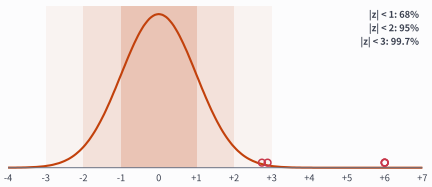
z は、平均より標準偏差いくつ分大きい（+）か小さい（−）か。
単位が違っても比べられる。

z says how many standard deviations a value is above (+) or below (−) the mean, so different units can be compared.

およそ正規分布なら、|z| < 1 に約 68%、|z| < 2 に約 95%、
|z| < 3 に約 99.7% が入る。

For roughly normal data, about 68% of values have |z| < 1, 95% have |z| < 2 and 99.7% have |z| < 3.

- 外れ値の目安：|z| ≥ 3（広めなら ≥ 2）、
  または Q1 − 1.5 × IQR より下・Q3 + 1.5 × IQR より上
  Outlier rules: |z| ≥ 3 (or ≥ 2), or below Q1 − 1.5 × IQR / above Q3 + 1.5 × IQR
- 外れ値は「間違い」ではなく「確かめる値」。
  入力ミスかまとめ買いかを確かめてから除く
  An outlier is a value to check, not a mistake: typing error or bulk order? Check before removing it



正規分布と ±1・±2・±3 の帯。
赤い丸は amount の z が大きい 5 件（右端の丸は z が 6 より大きい）。

The normal curve with the ±1, ±2, ±3 bands. Red circles: the 5 largest z of amount (circles at the right edge have z above 6).

見てみる See it

平均 5351.4、標準偏差 5214.0（A3・A4 で求めた値）を式に入れて z を計算し、大きい 5 件を出す。

Put the mean 5351.4 and the standard deviation 5214.0 (from A3 and A4) into the formula and list the 5 largest z.

```sql
SELECT order_id, amount,
       ROUND((amount - 5351.4) / 5214.0, 2) AS z
FROM kumo_orders
ORDER BY z DESC
LIMIT 5;
```

結果 5 行 5 rows

order_id	amount	z
88	68400	12.09
98	54200	9.37
265	47800	8.14
228	20430	2.89
368	19660	2.74

z が 5 を超える注文がある。入力ミスか、法人のまとめ買いか、伝票を確かめる。

Some orders have z above 5. Check the receipts: a typing error or a bulk order by a company?

ポイント Points

1. |z| ≥ 2 の件数は WHERE で数える。kumo_orders では 7 件。
   Count |z| ≥ 2 with WHERE: 7 orders in kumo_orders.
   `SELECT COUNT(*) FROM kumo_orders WHERE ABS(amount - 5351.4) / 5214.0 >= 2;`
2. 平均を同じ SQL の中で求めて使う方法（サブクエリ・ウィンドウ関数）は A9・A10 で学ぶ。
   Using the mean inside the same SQL (subqueries, window functions) comes in A9 and A10.

よくある間違い Common mistakes

✕
```
SELECT order_id, (amount - 5351) / 5214 AS z
FROM kumo_orders ORDER BY z DESC LIMIT 5;
```

実行結果 5 行 5 rows

order_id	z
~~88~~	~~12.09~~
~~98~~	~~9.37~~
~~265~~	~~8.14~~
~~228~~	~~2.89~~
~~368~~	~~2.74~~
88	12
98	9
265	8
92	2
141	2

→ 結果の値がちがう
→ different values in the result

整数だけの式は整数の割り算になり、z の小数が消える。小数点のある数を入れる。
An expression with only integers is an integer division and loses the decimals of z. Use numbers with a decimal point.

○
```
SELECT order_id, (amount - 5351.4) / 5214.0 AS z
FROM kumo_orders ORDER BY z DESC LIMIT 5;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 平均と標準偏差を確かめ、テーブルに z の列を足して、大きい順に並べ替える。
   Check the mean and standard deviation, add a z column and sort it from largest.

```
=ROUND(AVERAGE(kumo_orders[amount]),1)
=ROUND(STDEV.S(kumo_orders[amount]),1)
=ROUND(STANDARDIZE([@amount],5351.4,5214.0),2)
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. z 得点と、大きい 5 件・|z| ≥ 2 の件数：
   z-scores, the 5 largest and the count with |z| ≥ 2:

```
m, s = 5351.4, 5214.0
df["z"] = ((df["amount"] - m) / s).round(2)
print(df.nlargest(5, "z")[["order_id", "amount",
"z"]])
print((df["z"].abs() >= 2).sum())
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. helpdesk_tickets の minutes（平均 78.1、標準偏差 114.3）で、z が大きい 5 件の ticket_id と z を出そう。
   For minutes in helpdesk_tickets (mean 78.1, standard deviation 114.3), list ticket_id and z of the 5 largest z.
   結果：5 行 5 rows

2. factory_lots の weight_g（平均 501.8、標準偏差 10.9）で、|z| が 2 以上の件数を数えよう。
   For weight_g in factory_lots (mean 501.8, standard deviation 10.9), count the rows with |z| ≥ 2.
   結果：1 行 1 row

3. z = 1.5 の社員の偏差値はいくつ？
   What is the T-score (hensachi) of an employee with z = 1.5?
   計算で答える answer by calculation

分析 Unit A7

クロス集計と構成比

クロス集計・割合

Cross tables and shares

アプリで練習
Practice in the app

SQL Excel Python

販促担当 Promotions

「クーポンはどの店でよく使われている？」店ごとに数と割合で比べよう。

"In which store are coupons used most?" Compare stores by count and by share.

2 つの列で同時に数え、大きさのちがうグループを % で比べられる。

You can count by two columns at once and compare groups of different sizes in %.

統計の基礎 Statistics basics

クロス集計 Cross tables

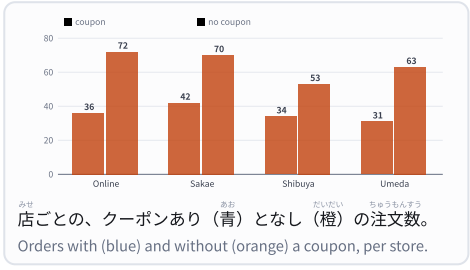
店 × クーポンのように、2 つのことがらで同時に行を数える。

「グループに違いはあるか」の第一歩。

Count rows by two things at once, like store × coupon: the first step to "do the groups differ?"

```
割合 (%) = 100.0 × 部分 ÷ 全体
```

- 行の割合：「渋谷の注文のうち 39% がクーポン」
  Row %: "39% of Shibuya's orders used a coupon"
- 全体に占める割合：「売上全体の 25% が渋谷」
  Share of the whole: "25% of all sales are Shibuya"
- 大きな店は何でも数が多い。比べるときは %
  Big stores have more of everything: compare in %



店ごとの、クーポンあり（青）となし（橙）の注文数。

Orders with (blue) and without (orange) a coupon, per store.

見てみる See it

店ごとに、注文数・クーポンを使った注文の数・その割合を出す。

For each store: orders, orders with a coupon, and that share.

```sql
SELECT store, COUNT(*) AS n,
       COUNT(coupon) AS with_coupon,
       ROUND(100.0 * COUNT(coupon) / COUNT(*), 1) AS pct_coupon
FROM kumo_orders
GROUP BY store
ORDER BY pct_coupon DESC;
```

結果 4 行 4 rows

store	n	with_coupon	pct_coupon
Shibuya	87	34	39.1
Sakae	112	42	37.5
Online	108	36	33.3
Umeda	94	31	33

割合がいちばん高いのは Shibuya（39.1%）。数だけ見ると別の店が多く見える。

Shibuya has the highest share (39.1%). By count alone another store looks bigger.

値ごとに列を分ける One column per value

```sql
SELECT store,
       SUM(CASE WHEN coupon = 'WELCOME10' THEN 1 ELSE 0 END) AS welcome10,
       SUM(CASE WHEN coupon = 'SPRING5' THEN 1 ELSE 0 END) AS spring5
FROM kumo_orders
GROUP BY store;
```

store	welcome10	spring5
Online	8	11
Sakae	5	10
Shibuya	6	12
Umeda	8	7

SUM(CASE WHEN … THEN 1 ELSE 0 END) は、条件に合う行だけを数える。Excel のピボットテーブルの「列」と同じ形。

SUM(CASE WHEN … THEN 1 ELSE 0 END) counts only the rows that meet the condition: the same shape as the Columns of an Excel PivotTable.

ポイント Points

1. COUNT(列) は NULL でない行を数える。クーポンの有無はこれで数えられる。
   COUNT(column) counts the non-NULL rows: that is how to count orders with a coupon.
2. 全体に占める割合は、(SELECT SUM(…) FROM …) で全体の合計を出して割る。
   For a share of the whole, divide by the grand total from (SELECT SUM(…) FROM …).

```
ROUND(100.0 * SUM(amount) / (SELECT SUM(amount) FROM kumo_orders), 1)
```

よくある間違い Common mistakes

✕

```
SELECT store, ROUND(100 * COUNT(coupon) / COUNT(*), 1) AS pct
FROM kumo_orders GROUP BY store;
```

実行結果 4 行 4 rows

store	pct
~~Online~~	~~33.3~~
~~Sakae~~	~~37.5~~
~~Shibuya~~	~~39.1~~
~~Umeda~~	~~33~~
Online	33
Sakae	37
Shibuya	39
Umeda	32

→ 結果の値がちがう
→ different values in the result

整数どうしの割り算は小数が消える。100.0 にする。
Integer division drops the decimals: use 100.0.

○

```
SELECT store, ROUND(100.0 * COUNT(coupon) / COUNT(*), 1) AS pct
FROM kumo_orders GROUP BY store;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 挿入 → ピボットテーブル：store を「行」、
   store と coupon を「値」（個数）に。横に割合を計算する。
   Insert → PivotTable: store to Rows, store and coupon to Values (Count). Add the share next to it.

```
=ROUND(100*C5/B5,1)
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 店ごとの数・クーポンあり・割合。
   pd.crosstab で値ごとの表も作れる。
   Count, with coupon and share per store. pd.crosstab makes the table per value.

```
s = df.groupby("store").agg(n=("coupon", "size"), with_coupon=("coupon", "count"))
s["pct_coupon"] = (100 * s["with_coupon"] / s["n"]).round(1)
print(s.sort_values("pct_coupon", ascending=False))
pd.crosstab(df["store"], df["coupon"])
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 gym_visits で branch ごとに、行数と trainer が入っている割合（%、小数第 1 位）を出そう。
For gym_visits, per branch: rows and the share (%, 1 decimal) with a trainer.

結果：3 行
3 rows

2 cafe_sales で shop ごとの amount の合計と、全体に占める割合を出そう。
For cafe_sales, the total amount per shop and its share of the whole.

結果：3 行
3 rows

3 helpdesk_tickets で team ごとに、rating が 'good' と 'bad' の件数を別の列で出そう。
For helpdesk_tickets, per team, count rating 'good' and 'bad' in separate columns.

結果：3 行
3 rows

分析 Unit A8

月ごとの推移 strftime・LAG・移動平均

Month by month

アプリで練習 Practice in the app

SQL Excel Python

店長 Store manager

「売上は落ちている？伸びている？」月ごとに並べて、前の月と比べよう。

"Are sales going down or up?" Put them month by month and compare with the month before.

日付を月にまとめ、前月との差と移動平均で流れを読める。

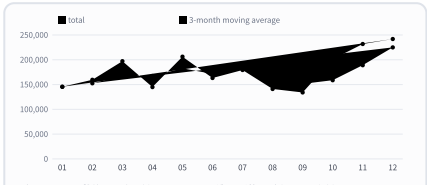
You can group dates into months and read the trend with the change from last month and a moving average.

統計の基礎 Statistics basics

時系列データ Time series

- 傾向：長い期間で上がるか下がるか
  Trend: up or down over a long time
- 季節性：毎年同じ形（12 月に売れる など）
  Seasonality: the same pattern every year (more sales in December…)
- ノイズ：たまたまの上がり下がり
  Noise: random ups and downs

差 = 今月 − 先月　　3 か月移動平均 = (今月 + 先月 + 先々月) ÷ 3



月ごとの合計（実線）と 3 か月移動平均（点線）。

Monthly totals (line) and the 3-month moving average (dashed).

移動平均はノイズをならして傾向を見やすくする。季節性があるときは前年の同じ月と比べる。

A moving average smooths the noise so the trend shows. With seasonality, compare with the same month last year.

見てみる See it

strftime('%Y-%m', order_date) で月にまとめ、月ごとの件数と売上合計を出す。

Group by month with strftime('%Y-%m', order_date): orders and total amount per month.

```sql
SELECT strftime('%Y-%m', order_date) AS month,
       COUNT(*) AS n, SUM(amount) AS total_amount
FROM kumo_orders
GROUP BY month
ORDER BY month;
```

結果 12 行　12 rows

month	n	total_amount
2025-01	29	145490
2025-02	36	159600
2025-03	26	197150
2025-04	20	144910
2025-05	39	206230
2025-06	37	163220
2025-07	33	179360
2025-08	29	141040
2025-09	21	134100
2025-10	39	201130
2025-11	45	231800
2025-12	47	241880

12 月に向けて伸びている。月による上がり下がりも大きい。

Sales grow towards December, with big ups and downs from month to month.

前の月と比べる（LAG） Compare with the month before (LAG)

```sql
SELECT month, total,
       total - LAG(total) OVER (ORDER BY month) AS diff
FROM (SELECT strftime('%Y-%m', order_date) AS month,
             SUM(amount) AS total
      FROM kumo_orders GROUP BY month)
ORDER BY diff DESC
LIMIT 3;
```

month	total	diff
2025-10	201130	67030
2025-05	206230	61320
2025-03	197150	37550

いちばん伸びたのは 2025-10（+67,030）。LAG(…) OVER (ORDER BY month) は月の順で 1 つ前の行の値。

The biggest rise was 2025-10 (+67,030). LAG(…) OVER (ORDER BY month) is the value of the month before.

ポイント Points

1. ウィンドウ関数（… OVER）は行をまとめない。行はそのままで列が増える。
   Window functions (… OVER) do not join rows: every row stays and gets a new column.
2. 3 か月移動平均：AVG(total) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
   3-month moving average: AVG(total) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

よくある間違い Common mistakes

✕
```
SELECT strftime('%Y %M', order_date) AS month, SUM(amount) AS total
FROM kumo_orders GROUP BY month;
```

実行結果 1 行 1 row

month	total
2025-01	145490
2025-02	159600
2025-03	197150
2025-04	144910

… 全部で 1 行

→ 1 行になる（正しくは 12 行）
→ 1 rows (correct: 12 rows)

%M（大文字）は「分」。月は %m（小文字）。全部が 1 行にまとまってしまう。
%M (capital) means minutes; the month is %m. Everything ends up in one row.

◯
```
SELECT strftime('%Y-%m', order_date) AS month, SUM(amount) AS total
FROM kumo_orders GROUP BY month;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. テーブルに month の列を足し、
   ピボットテーブルで月ごとに合計。挿入 → 折れ線グラフ。
   Add a month column, total it per month in a PivotTable, then Insert → Line chart.

```
=TEXT([@order_date],"yyyy-mm")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 月ごとの合計・前月との差・3 か月移動平均：
   Monthly total, change from last month, 3-month moving average:

```
s = df.groupby(df["order_date"].str[:7])
["amount"].sum().to_frame("total")
s["diff"] = s["total"].diff()
s["ma3"] = s["total"].rolling(3,
min_periods=1).mean().round(1)
s
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 courier_jobs の ship_date の月ごとに、件数と fee の合計を出そう。
For courier_jobs, per month of ship_date: jobs and total fee.
結果：12 行
12 rows

2 gym_visits の minutes を月ごとに合計し、前の月との差（diff）を出そう。
For gym_visits, total minutes per month and the change from the month before (diff).
結果：12 行
12 rows

3 hotel_stays の charge の月ごとの合計と、3 か月移動平均（小数第 1 位）を出そう。
For hotel_stays, the monthly total charge and the 3-month moving average (1 decimal).
結果：12 行
12 rows

分析 Unit A9

2つの数の関係（相関）

Relationship between two numbers

相関係数 r

アプリで練習
Practice in the app

SQL Excel Python

店舗企画 Store planning

「店に長くいるお客ほど、たくさん買う？」滞在時間と金額の関係を調べよう。

“Do customers who stay longer buy more?” Check the relationship between time in the store and amount.

散布図と相関係数で関係の強さを言え、相関と因果を区別できる。

You can describe the strength of a relationship with a scatter plot and r, and tell correlation from causation.

統計の基礎 Statistics basics

$$r = \Sigma(x - \bar{x})(y - \bar{y}) \div \sqrt{\left(\Sigma(x - \bar{x})^2 \times \Sigma(y - \bar{y})^2\right)}$$

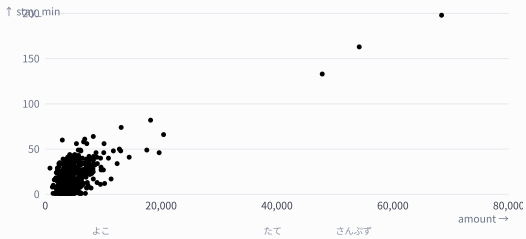
|r| ≥ 0.7 強い・0.4〜0.7 中くらい・0.2〜0.4 弱い・0.2 未満

- ほとんどない（目安）

  |r| ≥ 0.7 strong, 0.4–0.7 moderate, 0.2–0.4 weak, below 0.2 almost none (rule of thumb)
- r は直線的な関係だけを見る。必ず散布図も見る

  r sees only straight-line relationships: always look at the scatter plot
- 相関は因果ではない：アイスの売上と水の事故は、気温が両方を動かす

  Correlation is not causation: ice-cream sales and swimming accidents are both moved by the temperature



amount（横）と stay_min（縦）の散布図。点線はいちばん合う直線。

amount (x) and stay_min (y), with the best straight line (dashed).

見てみる See it

平均との差を掛け合わせて足し、両方の散らばりで割る＝相関係数。

Multiply the differences from the mean, add them up, and divide by both spreads: that is r.

```sql
SELECT COUNT(*) AS n,
  ROUND(SUM((amount - m.mx) * (stay_min - m.my))
    / SQRT(SUM((amount - m.mx) * (amount - m.mx))
        * SUM((stay_min - m.my) * (stay_min - m.my))), 3) AS r
FROM kumo_orders,
  (SELECT AVG(amount) AS mx, AVG(stay_min) AS my
   FROM kumo_orders) AS m;
```

結果 1行 1 row

n	r
401	0.77

r = 0.766：強い正の相関。ただし「長くいさせれば売れる」とは言えない（たくさん買う人は選ぶのに時間がかかる、かもしれない）。

r = 0.766: a strong positive correlation. But it does not show that keeping people longer sells more (people who buy a lot may need longer to choose).

ポイント Points

1. (SELECT AVG…) AS m を FROM に並べると、全部の行で m.mx・m.my が使える。

   Putting (SELECT AVG…) AS m in FROM makes m.mx and m.my usable on every row.
2. 少しの外れ値で r は大きく変わる。外れ値を除いた r とも比べる。

   A few outliers can change r a lot: compare with r without them.

よくある間違い Common mistakes

```
SELECT ROUND(SUM((amount - m.mx) * (stay_min - m.my)) / COUNT(*), 3) AS r
FROM kumo_orders, (SELECT AVG(amount) AS mx, AVG(stay_min) AS my FROM kumo_orders) AS m;
```

実行結果 1行 1 row

r
~~0.77~~
75299.97

→ 結果の値がちがう
→ different values in the result

行数で割ると「共分散」。単位に左右されるので、両方の散らばりで割って −1〜1 にする。
Dividing by the rows gives the covariance, which depends on the units. Divide by both spreads to get −1 to 1.

```
SELECT ROUND(SUM((amount - m.mx) * (stay_min - m.my)) / SQRT(SUM((amount - m.mx) *
(amount - m.mx)) * SUM((stay_min - m.my) * (stay_min - m.my))), 3) AS r
FROM kumo_orders, (SELECT AVG(amount) AS mx, AVG(stay_min) AS my FROM kumo_orders) AS m;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、
   名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. 相関係数。散布図は 2 列を選んで挿入 → 散布図、
   グラフ要素 → 近似曲線。
   r. For the scatter plot select both columns → Insert → Scatter, then Chart Elements → Trendline.

```
=ROUND(CORREL(kumo_orders[amount],kumo_orders[stay_min]),3)
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. 相関係数と散布図：
   r and the scatter plot:

```
print(round(df["amount"].corr(df["stay_min"]), 3))
df.plot.scatter(x="amount", y="stay_min")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 gym_visits の minutes と kcal の相関係数（小数第 3 位）を出そう。
Find r between minutes and kcal in gym_visits (3 decimals).
結果：1行 1 row

2 helpdesk_tickets の minutes と score の r を出し、符号（+・−）の意味を言おう。
Find r between minutes and score in helpdesk_tickets, and say what its sign means.
結果：1行 1 row

3 「雨の日は傘がよく売れ、客も少ない」。傘の売上と客数の相関は負。傘が客を減らした？
"On rainy days umbrellas sell well and there are fewer customers." Umbrella sales and customers correlate negatively. Did umbrellas reduce customers?
言葉で答える answer in words

分析 Unit A10

グループを比べる CASE・ROW_NUMBER

Comparing groups

アプリで練習
Practice in the app

SQL Excel Python

販促担当 Promotions

「クーポンを使った注文は大きい？」あり・なしで比べ、店ごとの一番も見よう。

“Are coupon orders bigger?” Compare with and without, and find the top order of each store.

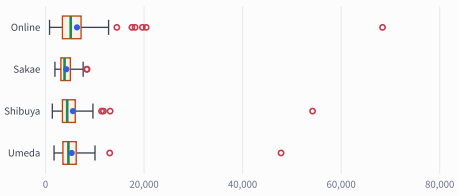
CASE で作ったグループを比べ、ROW_NUMBER でグループごとの順位を付けられる。

You can compare groups made with CASE and rank rows inside each group with ROW_NUMBER.

統計の基礎 Statistics basics

公平に比べる Comparing fairly

- 平均の差だけでなく、散らばり（箱ひげ図）と件数も見る
  Look at the spread (box plot) and the counts, not only the means
- 外れ値は平均を引っぱる：中央値も比べる
  Outliers pull the mean: compare medians too
- あり・なしの比較（A/B）は実験ではない。ほかの点でも違うかもしれない
  A with/without comparison is not an experiment: the groups may differ in other ways



店ごとの amount の箱ひげ図。
Box plot of amount per store.

見てみる See it

クーポンあり・なしで行を分け、それぞれの件数と平均金額を出す。

Split the orders into with and without a coupon: count and mean amount.

```sql
SELECT CASE WHEN coupon IS NULL THEN 'no' ELSE 'yes' END AS has_coupon,
       COUNT(*) AS n, ROUND(AVG(amount), 1) AS mean_amount
FROM kumo_orders
GROUP BY has_coupon
ORDER BY has_coupon;
```

結果 2 行 2 rows

has_coupon	n	mean_amount
no	258	5173.4
yes	143	5672.5

クーポンありの平均は 5,672.5、なしは 5,173.4。ただし「クーポンで増えた」とまでは言えない。

With a coupon the mean is 5,672.5, without 5,173.4. That does not prove the coupon caused it.

店ごとの一番（ROW_NUMBER） The top of each store (ROW_NUMBER)

```sql
SELECT store, order_id, amount
FROM (SELECT store, order_id, amount,
        ROW_NUMBER() OVER (PARTITION BY store ORDER BY amount DESC) AS rn
      FROM kumo_orders)
WHERE rn = 1
ORDER BY store;
```

store	order_id	amount
Online	88	68400
Sakae	215	8430
Shibuya	98	54200
Umeda	265	47800

PARTITION BY store で店ごとに 1, 2, 3 … と番号を付け直し、rn = 1 を残す。

PARTITION BY store numbers the rows again in each store; keep rn = 1.

ポイント Points

1. ウィンドウ関数は WHERE の中では使えない。( ) の中で番号を付けてから外側で絞る。
   Window functions cannot go in WHERE: number the rows in a subquery, then filter outside.
2. 「全体の平均より上の割合」も比べ方の 1 つ：CASE WHEN amount > (SELECT AVG(amount) ⋯) THEN 1 ELSE 0 END
   “Share above the overall mean” is another comparison: CASE WHEN amount > (SELECT AVG(amount) ⋯) THEN 1 ELSE 0 END

よくある間違い Common mistakes

✕
```
SELECT CASE WHEN coupon = NULL THEN 'no' ELSE 'yes' END AS has_coupon, COUNT(*) AS n
FROM kumo_orders GROUP BY has_coupon;
```

実行結果 1 行 1 row

has_coupon	n
~~no~~	~~258~~
~~yes~~	~~143~~
yes	401

→ 1 行になる（正しくは 2 行）
→ 1 rows (correct: 2 rows)

= NULL は決して成り立たない。NULL は IS NULL で調べる。
= NULL is never true. Test NULL with IS NULL.

○
```
SELECT CASE WHEN coupon IS NULL THEN 'no' ELSE 'yes' END AS has_coupon, COUNT(*) AS n
FROM kumo_orders GROUP BY has_coupon;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app’s CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. has_coupon の列を足し、ピボットテーブルで個数と平均。
   店ごとの最大は MAXIFS。
   Add a has_coupon column; Count and Average in a PivotTable. The largest per store: MAXIFS.

```
=IF([@coupon]="","no","yes")
=MAXIFS(kumo_orders[amount],kumo_orders[store],"Umeda")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. あり・なしの比較と、店ごとの最大の注文：
   With/without, and the largest order of each store:

```
df["has_coupon"] = df["coupon"].notna().map({True:
"yes", False: "no"})
print(df.groupby("has_coupon")["amount"].agg(["count",
"mean"]).round(1))
df.loc[df.groupby("store")["amount"].idxmax(),
["store", "order_id", "amount"]]
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. courier_jobs で note のある・なし（'yes'・'no'）ごとに、件数と fee の平均を出そう。
   For courier_jobs, with and without a note ('yes', 'no'): count and mean fee.
   結果：2 行 2 rows

2. hotel_stays で hotel ごとに、charge がいちばん大きい宿泊の stay_id と charge を出そう。
   For hotel_stays, per hotel: stay_id and charge of the largest charge.
   結果：3 行 3 rows

3. gym_visits で branch ごとに、minutes が全体の平均より長い割合（%）を出そう。
   For gym_visits, per branch: the share (%) of visits longer than the overall mean.
   結果：3 行 3 rows

分析 Unit A11

2 つの表を合わせて分析する

JOIN して集計

Analysing two tables together

アプリで練習
Practice in the app

SQL Excel Python

会員担当 Membership

「Gold 会員は本当にたくさん買っている？」注文と会員の表を合わせよう。

“Do Gold members really buy more?” Combine the orders and the member tables.

人のコードで JOIN し、人の表の列でグループにして集計できる。

You can JOIN on the person’s code and group by a column of the people table.

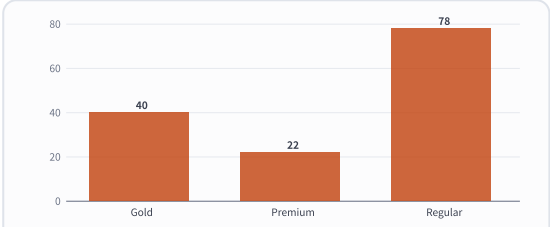
統計の基礎 Statistics basics

何が起きたか（注文）と、誰がしたか（会員）は別の表にある。

customer_id でつなぐ。

What happened (orders) and who did it (members) are in different tables. Join them on customer_id.

- INNER JOIN：両方に相手がいる行だけ
  INNER JOIN: only rows with a partner in both
- LEFT JOIN … WHERE 右.id IS NULL：一度も買っていない人
  LEFT JOIN … WHERE right.id IS NULL: people who never bought
- JOIN の後は数え方に注意：注文の多い人はたくさんの行になる
  Count with care after a JOIN: a person with many orders becomes many rows



kumo_accounts のプランごとの人数。

People per plan in kumo_accounts.

見てみる See it

kumo_orders と kumo_accounts を customer_id でつなぎ、プランごとに注文数・合計・平均を出す。

Join kumo_orders and kumo_accounts on customer_id: orders, total and mean per plan.

```sql
SELECT kumo_accounts.plan, COUNT(*) AS n,
       SUM(kumo_orders.amount) AS total_amount,
       ROUND(AVG(kumo_orders.amount), 1) AS mean_amount
FROM kumo_orders
  INNER JOIN kumo_accounts
  ON kumo_orders.customer_id = kumo_accounts.customer_id
GROUP BY kumo_accounts.plan
ORDER BY total_amount DESC;
```

結果 3 行 3 rows

plan	n	total_amount	mean_amount
Regular	235	1279430	5444.4
Gold	108	557970	5166.4
Premium	58	308510	5319.1

合計は Regular がいちばん。でも平均で見ると、プランの差は小さい。合計の差は人数の差。

Regular has the largest total, but the means are close: the difference in totals comes from the number of people.

ポイント Points

1. 同じ名前の列（customer_id）が両方にあるので、表名.列名 で書く。
   customer_id is in both tables, so write table.column.
2. 人ごとに GROUP BY すると、購入回数や合計で上位のお客が分かる（RFM 分析の入り口）。
   GROUP BY person shows the best customers by number and total of purchases (the start of RFM analysis).

よくある間違い Common mistakes

✕
```
SELECT plan, COUNT(*) AS n
FROM kumo_accounts GROUP BY plan;
```

実行結果 3 行 3 rows

plan	n
~~Gold~~	~~108~~
~~Premium~~	~~58~~
~~Regular~~	~~235~~
Gold	40
Premium	22
Regular	78

→ 結果の値がちがう
→ different values in the result

会員の表だけでは「人数」しか数えられない。注文を数えるには JOIN する。
The member table alone counts people. To count orders, JOIN.

○
```
SELECT kumo_accounts.plan, COUNT(*) AS n
FROM kumo_orders INNER JOIN kumo_accounts
  ON kumo_orders.customer_id = kumo_accounts.customer_id
GROUP BY kumo_accounts.plan;
```

X Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. kumo_accounts もテーブルにし、
   注文にプランを持ってくる。そのあとピボットテーブル。
   Make kumo_accounts a table too, bring the plan into the orders, then a PivotTable.

```
=XLOOKUP([@customer_id],kumo_accounts[customer_id],k
umo_accounts[plan])
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Py Python でやってみる

Python (pandas・VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、
   pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. merge でつないで、プランごとに集計：
   merge, then aggregate per plan:

```
acc = pd.read_csv("kumo_accounts.csv")
m = df.merge(acc, on="customer_id")
m.groupby("plan")["amount"].agg(["count", "sum",
"mean"]).round(1)
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 gym_visits と gym_members を member_id でつなぎ、plan ごとの利用回数と minutes の合計を出そう。
Join gym_visits and gym_members on member_id: visits and total minutes per plan.
結果：3 行 3 rows

2 cafe_members のうち、cafe_sales に一度も出てこない人を plan ごとに数えよう。
Count the cafe_members who never appear in cafe_sales, per plan.
結果：3 行 3 rows

3 courier_jobs で client_id ごとの件数と fee の合計を出し、合計の大きい 3 社を出そう。
For courier_jobs, jobs and total fee per client_id: the top 3 by total.
結果：3 行 3 rows

分析
Unit
A12

データの下ごしらえ CASE・TRIM・COALESCE

Getting data ready

アプリで練習
Practice in the app

SQL Excel Python

会員担当
Membership

「都道府県ごとの会員数を出して。」でも手入力の列は書き方がばらばら。

"Members per prefecture, please." But the hand-typed column is written in many ways.

書き方のゆれをそろえ、空欄に名前を付け、数を区分にまとめられる。

You can unify spellings, name the blanks and put numbers into bands.

統計の基礎 Statistics basics

分析の時間の多くは準備にかかる。まず数えて、値を見て、それから整える。
元の列は残し、新しい列を作る。

Most analysis time goes into preparation. Count, look at the values, then tidy. Keep the original column and make a new one.

- 書き方のゆれ：TRIM（前後の空白）・LOWER／UPPER（大文字小文字）・REPLACE
  Spelling: TRIM (spaces), LOWER / UPPER (case), REPLACE
- 空欄：COALESCE(x, '(none)') ではっきり見せる。数をだまって 0 にしない
  Blanks: show them with COALESCE(x, '(none)'). Never silently put 0 in numbers
- 区分：CASE WHEN で年代・価格帯。区切りはきりのよい数に
  Bands: CASE WHEN for age or price bands, with round numbers as edges

見てみる See it

pref の前後の空白を消して小文字にし、都道府県ごとに数える。

Remove the spaces around pref, make it lower case, and count per prefecture.

```sql
SELECT LOWER(TRIM(pref)) AS pref_clean, COUNT(*) AS n
FROM kumo_accounts
GROUP BY pref_clean
ORDER BY n DESC;
```

結果 5 行 5 rows

pref_clean	n
saitama	30
tokyo	29
osaka	28
aichi	27
kanagawa	26

元の pref は 22 通りの書き方があったが、整えると 5 つになる。

pref was written in 22 different ways; after tidying there are 5.

年代に分ける（CASE WHEN） Age bands (CASE WHEN)

```sql
SELECT CASE WHEN age < 30 THEN '1: < 30'
            WHEN age < 50 THEN '2: 30-49'
            ELSE '3: 50+' END AS band,
       COUNT(*) AS n
FROM kumo_accounts
GROUP BY band
ORDER BY band;
```

band	n
1: < 30	26
2: 30-49	75
3: 50+	39

条件は上から順に調べられ、最初に合ったものになる。名前の先頭に番号を付けると順番どおりに並ぶ。

The conditions are checked from the top; the first match wins. A number at the start of each name keeps the bands in order.

ポイント Points

1. 数える前に GROUP BY で値の一覧を見る。おかしな書き方はここで見つかる。
   Before counting, list the values with GROUP BY: odd spellings show up here.
2. COALESCE(coupon, '(none)') で空欄も 1 つのグループとして数えられる。
   COALESCE(coupon, '(none)') counts the blanks as one group too.

よくある間違い Common mistakes

✕

```
SELECT pref, COUNT(*) AS n
FROM kumo_accounts GROUP BY pref;
```

実行結果 22 行 22 rows

pref	n
~~aichi~~	~~27~~
~~kanagawa~~	~~26~~
~~osaka~~	~~28~~
~~saitama~~	~~30~~
~~tokyo~~	~~29~~
Aichi	2

… 全部で 22 行

→ 22 行になる（正しくは 5 行）
→ 22 rows (correct: 5 rows)

そのままでは ' tokyo' と 'Tokyo' が別の県として数えられる。
As it is, ' tokyo' and 'Tokyo' are counted as different places.

○

```
SELECT LOWER(TRIM(pref)) AS pref_clean, COUNT(*) AS n
FROM kumo_accounts GROUP BY pref_clean;
```

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。
   Ctrl + T でテーブルにして、名前を kumo_accounts にする。
   Save the table with the app's CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_accounts.
2. 整えた列と年代の列を足して、ピボットテーブルで数える。
   Add a tidy column and a band column, then count in a PivotTable.

```
=LOWER(TRIM([@pref]))
=IFS([@age]<30,"1: < 30",[@age]<50,"2: 30-49",TRUE,"3: 50+")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_accounts.csv")
```

2. 整える前と後、年代：
   Before and after, and the bands:

```
print(df["pref"].value_counts())
df["pref_clean"] = df["pref"].str.strip().str.lower()
print(df["pref_clean"].value_counts())
import numpy as np
df["band"] = np.select([df["age"] < 30, df["age"] < 50],
["1: < 30", "2: 30-49"], "3: 50+")
df.groupby("band").size()
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1. gym_members の pref を整えて、都道府県ごとの人数を多い順に出そう。
   Tidy pref in gym_members and count members per prefecture, most first.
   結果：5 行 5 rows

2. hotel_stays の request の空欄を '(none)' として、値ごとの件数を出そう。
   In hotel_stays, show blank request as '(none)' and count each value.
   結果：5 行 5 rows

3. courier_clients の employees を 50 未満・50〜99・100 以上に分けて、社数を数えよう。
   Put employees in courier_clients into < 50, 50–99 and 100+, and count the clients.
   結果：3 行 3 rows

分析
Unit
A13

結果を正しく読む

Reading results correctly

結果を読む

アプリで練習
Practice in the app

SQL

部長
Manager

「この数字から何が言える？」計算より読み取りの間違いのほうが危ない。

“What does this number tell us?” Wrong readings are more dangerous than wrong calculations.

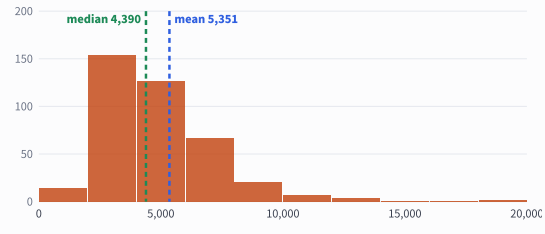
代表値・相関の強さ・数と割合・因果・外れ値について、正しい読み方を選べる。

You can choose the right reading of typical values, correlation strength, counts vs shares, causation and outliers.

統計の基礎 Statistics basics

数字を信じる前の 5 つの問い Five questions before you believe a number

- ふつうの値：平均？中央値？（とても大きな値はない？）
  Typical: mean or median? (Any very large values?)
- 数？割合？（グループの大きさは同じ？）
  Count or share? (Same group sizes?)
- どれくらい強い？（r と散布図）
  How strong? (r and the scatter plot)
- 原因？いっしょに動くだけ？（第 3 のものはない？）
  Cause, or only together? (A third thing?)
- めずらしい？ふつう？（z はいくつ？）
  Unusual or normal? (What is z?)



amount の分布。平均は大きな値に引っぱられ、中央値より右にある。

The distribution of amount. The mean is pulled right by large values.

見てみる See it

amount の平均・中央値・最大値。「ふつうの注文」を表すのはどれ？

The mean, median and largest amount. Which shows a “typical order”?

```sql
SELECT ROUND(AVG(amount), 1) AS mean_amount,
       (SELECT amount FROM kumo_orders
        ORDER BY amount LIMIT 1 OFFSET 200) AS median_amount,
       MAX(amount) AS max_amount
FROM kumo_orders;
```

結果 1 行 1 row

mean_amount	median_amount	max_amount
5351.4	4390	68400

最大 68,400 のような大きな注文が平均を引っぱる。ふつうの注文は中央値 4,390 のほうがよく表す。

Big orders like 68,400 pull the mean up. The median 4,390 shows a typical order better.

ポイント Points

1. 行数のいちばん多いグループが、割合でもいちばんとは限らない。
   The group with the most rows is not always the one with the largest share.
2. 合計がいちばん大きい月と、いちばん伸びた月は別。
   The month with the largest total is not the month that grew the most.
3. 相関は「いっしょに動く」だけ。原因を言うには実験などの別の証拠がいる。
   Correlation only means “move together”. A cause needs other evidence such as an experiment.

読んでみよう Read it

1. A7 の結果で、件数がいちばん多い店と、クーポンの割合がいちばん高い店は同じ？
   In the A7 result, is the store with the most orders the store with the highest coupon share?
   — A7 の表を見る / look at the A7 table

2. r = −0.77 のとき、「片方が大きいともう片方は？」と、その強さを言おう。
   r = −0.77: when one is large, what about the other, and how strong is it?
   — 言葉で答える / answer in words

3. z が 2.4 の注文は、|z| ≥ 3 の目安で外れ値？
   Is an order with z = 2.4 an outlier by the |z| ≥ 3 rule?
   — はい／いいえと理由 / yes / no and why

分析 Unit A14

次のキャンペーンはどの店で？

総合演習

Where should the next campaign be?

アプリで練習
Practice in the app

SQL Excel Python

社長 CEO

「クーポンを１つの店で配る。どこがいい？」学んだことを使い、数字を理由に決めよう。

“We will hand out coupons in one store. Which one?” Use what you learned and decide, with the numbers as your reason.

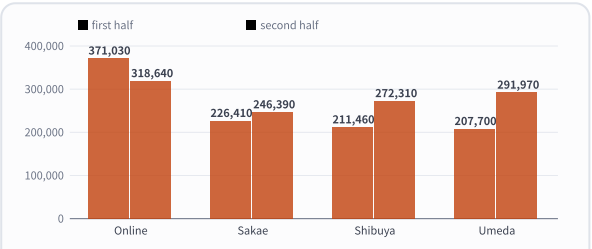
問い → 全体像 → 比較 → 人 → 決定 の流れで分析し、１文で報告できる。

You can go from question → overview → comparison → people → decision, and report it in one sentence.

統計の基礎 Statistics basics

- 1. 全体像：店ごとの件数・合計・平均（A1・A3）
  1. Overview: count, total, mean per store (A1, A3)
- 2. 今のクーポン利用率（A7）
  2. Coupon use now (A7)
- 3. 上半期と下半期の売上（A8）
  3. Sales in the first and second half (A8)
- 4. クーポンあり・なしの平均（A10）
  4. Mean with and without a coupon (A10)
- 5. どんなお客か（A11）→ 6. 決める
  5. Who the customers are (A11) → 6. Decide

報告 ＝ 結論 ＋ 数字の理由 ＋ 気をつけること



店ごとの上半期（青）と下半期（橙）の売上。

Sales per store, first half (blue) and second half (orange).

見てみる See it

決めるための表：店ごとのクーポン利用率と、下半期 − 上半期 の売上の伸び。

The table for the decision: coupon share and growth (second half − first half) per store.

```sql
SELECT store,
       ROUND(100.0 * COUNT(coupon) / COUNT(*), 1) AS pct_coupon,
       SUM(CASE WHEN order_date >= '2025-07-01' THEN amount ELSE 0 END)
     - SUM(CASE WHEN order_date <  '2025-07-01' THEN amount ELSE 0 END) AS
growth
FROM kumo_orders
GROUP BY store
ORDER BY store;
```

結果 4 行 4 rows

store	pct_coupon	growth
Online	33.3	-52390
Sakae	37.5	19980
Shibuya	39.1	60850
Umeda	33	84270

伸びている店（growth > 0）のうち、クーポンの割合がいちばん低いのは Umeda（33.0%）。

Of the growing stores (growth > 0), Umeda has the lowest coupon share (33.0%).

ポイント Points

1. 報告の例：「クーポンは Umeda で配る。下半期に売上が伸びているのに、クーポンを使う注文は 33.0% しかない。平均金額が下がらないか見ていく。」
   Example report: “Hand out coupons in Umeda: its sales are growing but only 33.0% of orders use a coupon. We will watch whether the mean amount falls.”
2. 決め方（基準）を先に言う。数字を見てから基準を変えない。
   State the rule for deciding first. Do not change it after seeing the numbers.

Excel でやってみる Excel (Microsoft 365)

1. アプリの「CSV」ボタンで表を保存し、Excel で開く。Ctrl + T でテーブルにして、名前を kumo_orders にする。
   Save the table with the app’s CSV button and open it in Excel. Press Ctrl + T to make a table and name it kumo_orders.
2. half の列を足し、ピボットテーブル：store を「行」、half を「列」、amount の合計。
   Add a half column; PivotTable: store in Rows, half in Columns, Sum of amount.

```
=IF([@order_date]
<DATE(2025,7,1),"first_half","second_half")
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

Python でやってみる

Python (pandas · VS Code / Jupyter)

1. CSV をノートブックと同じフォルダに置き、pandas で読み込む。
   Put the CSV in the same folder as your notebook and read it with pandas.

```
import pandas as pd
df = pd.read_csv("kumo_orders.csv")
```

2. クーポンの割合と伸び：
   Coupon share and growth:

```
second = df["order_date"] >= "2025-07-01"
s = df.groupby("store").agg(pct_coupon=
("coupon", lambda x: round(100 *
x.notna().mean(), 1)))
s["growth"] = df[second].groupby("store")
["amount"].sum() -
df[~second].groupby("store")["amount"].sum()
s
```

SQL と同じ数字になれば正解。
If you get the same numbers as the SQL, you are right.

やってみよう Try it

1 cafe_sales で、shop ごとに上半期（2025-07-01 より前）と下半期の amount の合計を出そう。
For cafe_sales, the total amount per shop in the first half (before 2025-07-01) and the second half.

結果：3 行
3 rows

2 cafe_sales で、shop ごとのクーポン利用率（%）を低い順に出そう。
For cafe_sales, the coupon share (%) per shop, lowest first.

結果：3 行
3 rows

3 1・2 の結果から、カフェで「次にクーポンを配る店」を 1 文で報告しよう。
From 1 and 2, report in one sentence which cafe should get the coupons next.

結論＋数字の理由
conclusion + reason with numbers

やってみようの答え Answers to “Try it”

結果は上の 6 行まで。別の書き方でも、同じ結果なら正解。

Results show up to 6 rows. A different SQL with the same result is also right.

A1-1

```sql
SELECT COUNT(*) AS n_rows, MIN(amount) AS min_amount, MAX(amount) AS max_amount FROM cafe_sales;
```

n_rows	min_amount	max_amount
151	200	9800

A1-2

```sql
SELECT MIN(checkin_date) AS first_date, MAX(checkin_date) AS last_date FROM hotel_stays;
```

first_date	last_date
2025-01-02	2025-12-23

A1-3 ID：visit_id ／ 日付：visit_date ／ 質的：branch・member_id・trainer ／ 量的：minutes・machines・kcal

ID: visit_id / date: visit_date / qualitative: branch, member_id, trainer / quantitative: minutes, machines, kcal

A2-1

```sql
SELECT COUNT(*) - COUNT(rating) AS missing, ROUND(100.0 * (COUNT(*) - COUNT(rating)) / COUNT(*), 1) AS missing_pct FROM helpdesk_tickets;
```

missing	missing_pct
42	34.7

A2-2

```sql
SELECT COUNT(DISTINCT client_id) AS clients FROM courier_jobs;
```

clients
48

A2-3

```sql
SELECT COUNT(DISTINCT member_id) AS members, COUNT(*) AS n_rows FROM cafe_sales;
```

members	n_rows
55	151

A3-1

```sql
SELECT ROUND(AVG(charge), 1) AS mean_charge, (SELECT charge FROM hotel_stays ORDER BY charge LIMIT 1 OFFSET 65) AS median_charge FROM hotel_stays;
```

mean_charge	median_charge
35812.2	28600

A3-2

```sql
SELECT minutes FROM gym_visits ORDER BY minutes LIMIT 1 OFFSET 80;
```

minutes
65

A3-3

```sql
SELECT defects, COUNT(*) AS n FROM factory_lots GROUP BY defects ORDER BY n DESC LIMIT 1;
```

defects	n
1	43

A4-1

```sql
SELECT team, ROUND(SQRT((SUM(minutes * minutes) - 1.0 * SUM(minutes) * SUM(minutes) / COUNT(*)) / (COUNT(*) - 1)), 1) AS sd_minutes FROM helpdesk_tickets GROUP BY team;
```

team	sd_minutes
Accounts	21.6
Network	70
Software	202.4

A4-2

```sql
SELECT shop, MAX(amount) - MIN(amount) AS range_amount FROM cafe_sales GROUP BY shop;
```

shop	range_amount
Campus	1170
Park	7400
Station	9470

A4-3

```sql
SELECT line, ROUND(SQRT((SUM(weight_g * weight_g) - 1.0 * SUM(weight_g) * SUM(weight_g) / COUNT(*)) / (COUNT(*) - 1)), 1) AS s FROM factory_lots GROUP BY line ORDER BY s;
```

line	s
Line A	3.8
Line B	10
Line C	15.7

A5-1

```sql
SELECT FLOOR(1.0 * fee / 500) * 500 AS bin, COUNT(*) AS n FROM courier_jobs GROUP BY bin ORDER BY bin;
```

bin	n
500	24
1000	57
1500	37
2000	16
2500	3
3500	2

… 全部で 8 行

A5-2

```sql
SELECT nights, COUNT(*) AS n, ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM hotel_stays), 1) AS pct FROM hotel_stays GROUP BY nights ORDER BY nights;
```

nights	n	pct
1	31	23.7
2	29	22.1
3	36	27.5
4	27	20.6
5	7	5.3
6	1	0.8

A5-3

```sql
SELECT FLOOR(1.0 * weight_g / 5) * 5 AS bin, COUNT(*) AS n FROM factory_lots GROUP BY bin ORDER BY bin;
```

bin	n
440	1
470	1
480	1
485	3
490	10
495	18

… 全部で 12 行

A6-1

```sql
SELECT ticket_id, ROUND((minutes - 78.1) / 114.3, 2) AS z FROM helpdesk_tickets ORDER BY z DESC LIMIT 5;
```

ticket_id	z
94	7.72
43	5.62
69	2.2
26	1.82
102	1.53

A6-2

```sql
SELECT COUNT(*) AS outliers FROM factory_lots WHERE ABS(weight_g - 501.8) / 10.9 >= 2;
```

outliers
5

A6-3

偏差値 = 50 + 10 × 1.5 = 65

T-score = 50 + 10 × 1.5 = 65

A7-1

```sql
SELECT branch, COUNT(*) AS n, ROUND(100.0 * COUNT(trainer) / COUNT(*), 1)
AS pct_trainer FROM gym_visits GROUP BY branch ORDER BY pct_trainer DESC;
```

branch	n	pct_trainer
Ikebukuro	62	51.6
Shinjuku	57	40.4
Kawasaki	42	33.3

A7-2

```sql
SELECT shop, SUM(amount) AS total_amount, ROUND(100.0 * SUM(amount)
/ (SELECT SUM(amount) FROM cafe_sales), 1) AS share_pct FROM
cafe_sales GROUP BY shop ORDER BY share_pct DESC;
```

shop	total_amount	share_pct
Park	76710	46.1
Station	52540	31.6
Campus	37000	22.3

A7-3

```sql
SELECT team, SUM(CASE WHEN rating = 'good' THEN 1 ELSE 0 END) AS good, SUM(CASE
WHEN rating = 'bad' THEN 1 ELSE 0 END) AS bad FROM helpdesk_tickets GROUP BY team
ORDER BY team;
```

team	good	bad
Accounts	6	7
Network	11	11
Software	9	9

A8-1

```sql
SELECT strftime('%Y-%m', ship_date) AS month, COUNT(*) AS n, SUM(fee) AS
total_fee FROM courier_jobs GROUP BY month ORDER BY month;
```

month	n	total_fee
2025-01	7	10250
2025-02	10	21060
2025-03	12	17060
2025-04	8	11150
2025-05	13	18020
2025-06	5	6920

… 全部で 12 行

A8-2

```sql
SELECT month, total, total - LAG(total) OVER (ORDER BY month) AS diff FROM
(SELECT strftime('%Y-%m', visit_date) AS month, SUM(minutes) AS total FROM
gym_visits GROUP BY month) ORDER BY month;
```

month	total	diff
2025-01	785	*NULL*
2025-02	1259	474
2025-03	1228	-31
2025-04	1449	221
2025-05	383	-1066
2025-06	1081	698

… 全部で 12 行

A8-3

```sql
SELECT month, total, ROUND(AVG(total) OVER (ORDER BY month ROWS BETWEEN 2
PRECEDING AND CURRENT ROW), 1) AS ma3 FROM (SELECT strftime('%Y-%m',
checkin_date) AS month, SUM(charge) AS total FROM hotel_stays GROUP BY month)
ORDER BY month;
```

month	total	ma3
2025-01	128500	128500
2025-02	318600	223550
2025-03	443800	296966.7
2025-04	474900	412433.3
2025-05	293600	404100
2025-06	222000	330166.7

… 全部で 12 行

A9-1

```sql
SELECT ROUND(SUM((minutes - m.mx) * (kcal - m.my)) / SQRT(SUM((minutes - m.mx) * (minutes - m.mx)) * SUM((kcal - m.my) * (kcal - m.my))), 3) AS r FROM gym_visits, (SELECT AVG(minutes) AS mx, AVG(kcal) AS my FROM gym_visits) AS m;
```

r
0.71

A9-2

```sql
SELECT ROUND(SUM((minutes - m.mx) * (score - m.my)) / SQRT(SUM((minutes - m.mx) * (minutes - m.mx)) * SUM((score - m.my) * (score - m.my))), 3) AS r FROM helpdesk_tickets, (SELECT AVG(minutes) AS mx, AVG(score) AS my FROM helpdesk_tickets) AS m;
```

r
-0.77

A9-3 いいえ。雨（第３のもの）が傘の売上を増やし、客を減らしている。相関は因果ではない。

No. The rain (a third thing) raises umbrella sales and lowers customers. Correlation is not causation.

A10-1

```sql
SELECT CASE WHEN note IS NULL THEN 'no' ELSE 'yes' END AS has_note, COUNT(*) AS n, ROUND(AVG(fee), 1) AS mean_fee FROM courier_jobs GROUP BY has_note ORDER BY has_note;
```

has_note	n	mean_fee
no	109	1519.1
yes	32	1658.1

A10-2

```sql
SELECT hotel, stay_id, charge FROM (SELECT hotel, stay_id, charge, ROW_NUMBER() OVER (PARTITION BY hotel ORDER BY charge DESC) AS rn FROM hotel_stays) WHERE rn = 1 ORDER BY hotel;
```

hotel	stay_id	charge
Kyoto	97	268000
Naha	5	132900
Sapporo	32	312000

A10-3

```sql
SELECT branch, ROUND(100.0 * SUM(CASE WHEN minutes > (SELECT AVG(minutes) FROM gym_visits) THEN 1 ELSE 0 END) / COUNT(*), 1) AS pct_above FROM gym_visits GROUP BY branch ORDER BY pct_above DESC;
```

branch	pct_above
Ikebukuro	51.6
Shinjuku	42.1
Kawasaki	26.2

A11-1

```sql
SELECT gym_members.plan, COUNT(*) AS n, SUM(gym_visits.minutes) AS total_minutes FROM gym_visits INNER JOIN gym_members ON gym_visits.member_id = gym_members.member_id GROUP BY gym_members.plan ORDER BY total_minutes DESC;
```

plan	n	total_minutes
Full	108	7397
Night	37	2627
Day	16	1093

A11-2

```sql
SELECT cafe_members.plan, COUNT(*) AS n FROM cafe_members LEFT JOIN cafe_sales ON cafe_members.member_id = cafe_sales.member_id WHERE cafe_sales.sale_id IS NULL GROUP BY cafe_members.plan ORDER BY cafe_members.plan;
```

plan	n
Basic	2
Gold	2
Student	1

A11-3

```sql
SELECT client_id, COUNT(*) AS n, SUM(fee) AS total_fee FROM courier_jobs GROUP BY client_id ORDER BY total_fee DESC LIMIT 3;
```

client_id	n	total_fee
K001	5	13760
K028	3	11380
K016	7	10890

A12-1

```sql
SELECT LOWER(TRIM(pref)) AS pref_clean, COUNT(*) AS n FROM gym_members GROUP BY pref_clean ORDER BY n DESC;
```

pref_clean	n
kanagawa	14
tokyo	12
saitama	11
osaka	10
aichi	8

A12-2

```sql
SELECT COALESCE(request, '(none)') AS request_filled, COUNT(*) AS n FROM hotel_stays GROUP BY request_filled ORDER BY n DESC;
```

request_filled	n
(none)	52
late check-in	24
extra bed	21
quiet room	17
airport bus	17

A12-3

```sql
SELECT CASE WHEN employees < 50 THEN '1: < 50' WHEN employees < 100 THEN '2: 50-99' ELSE '3: 100+' END AS band, COUNT(*) AS n FROM courier_clients GROUP BY band ORDER BY band;
```

band	n
1: < 50	15
2: 50-99	26
3: 100+	9

A13-1 同じではない。件数がいちばん多いのは Sakae（112 件）、クーポンの割合がいちばん高いのは Shibuya（39.1%）。

No. Most orders: Sakae (112); highest coupon share: Shibuya (39.1%).

A13-2 片方が大きいと、もう片方は小さい傾向（負の相関）。$|r| \geq 0.7$ なので強い。

When one is large the other tends to be small (negative). $|r| \geq 0.7$, so it is strong.

A13-3 いいえ。$|z| = 2.4$ は 3 より小さい。めずらしいが、外れ値の候補ではない。

No. $|z| = 2.4$ is below 3: unusual, but not an outlier candidate.

A14-1

```sql
SELECT shop, SUM(CASE WHEN sale_date < '2025-07-01' THEN amount
ELSE 0 END) AS first_half, SUM(CASE WHEN sale_date >= '2025-07-01'
THEN amount ELSE 0 END) AS second_half FROM cafe_sales GROUP BY
shop ORDER BY shop;
```

shop	first_half	second_half
Campus	16210	20790
Park	33790	42920
Station	27070	25470

A14-2

```sql
SELECT shop, ROUND(100.0 * COUNT(coupon) / COUNT(*), 1) AS pct_coupon FROM
cafe_sales GROUP BY shop ORDER BY pct_coupon;
```

shop	pct_coupon
Station	34.8
Campus	44.4
Park	45

A14-3 例：「クーポンは Campus で配る。下半期に売上が 4,580 円伸びているのに、クーポンの利用率は 44.4% で、伸びている店の中でいちばん低い。」

Example: “Hand out coupons at Campus: its sales grew by 4,580 yen in the second half, but its coupon share (44.4%) is the lowest of the growing shops.”